SHIFT — Software Requirements Specification (SRS)

Version 1.0 | Graduate Project — Helwan University, CS Department

---

TABLE OF CONTENTS

---

1. INTRODUCTION

1.1 Purpose

This document specifies the complete functional and non-functional requirements for **SHIFT**, a narrative-driven desktop-simulation game designed to teach introductory computer science concepts (Helwan CS111/COM101 curriculum) through situated workplace learning, stealth assessment, and adaptive practice gates.

1.2 Scope

SHIFT is a web-based game accessed via browser, simulating a desktop operating system ("LoopOS"). Players assume the role of Mohamed, a junior developer at a Cairo software studio, learning C programming through narrative decision-making, hands-on coding, and consequence-rich practice. The system includes a runtime AI pipeline for dynamic side-project task generation, a comprehensive economy system, and full administrative dashboards for instructors and developers.

1.3 Definitions & Acronyms

Term	Definition
Shift	A narrative "workday" unit containing story beats, choice points, and practice gates
Beat	A single narrative delivery unit (chat message, email, notification)
Tier	Choice quality level: Ideal, Acceptable, Debt, Mistake
Stealth Assessment	Embedded learning measurement through player behavior, not explicit tests

Table 1 – continued

Term	Definition
Practice Gate	Mandatory post-shift coding exercise with mastery-based exit condition
Side Project	Parallel self-directed coding tasks (Khamis brand tool) for income and extra practice
Sahm	AI coding assistant for side project tasks
LoopOS	Diegetic desktop operating system that constitutes the game UI
ECD	Evidence-Centered Design (assessment framework)
SDT	Self-Determination Theory (motivation framework)

1.4 References

- CS111/COM101 Curriculum (Dr. Amr S. Ghoneim)
- SHIFT Game Design Document v2.0
- SHIFT Literature Review Source Inventory (123 sources)
- IEEE 830-1998: Recommended Practice for Software Requirements Specifications

1.5 Document Overview

This SRS is organized into functional requirements (Section 3), interface requirements (Section 4), non-functional requirements (Section 5), data requirements (Section 6), security (Section 7), admin panels (Section 8), and AI pipeline (Section 9).

---

2. OVERALL DESCRIPTION

2.1 Product Perspective

SHIFT is a standalone web application with three integrated subsystems:

1. **Game Client** (React frontend) — Player-facing LoopOS desktop
2. **Game Server** (.NET backend monolith) — Auth, content, progress, economy, assessment
3. **AI Orchestration Service** (Python runtime) — Dynamic side-task generation via LLM API

2.2 User Classes & Characteristics

User Class	Description	Count (Pilot)	Technical Skill
Player (Student)	CS111 students playing the game	~50	Low-Medium
Admin (Instructor)	Dr. Amr and TAs monitoring student progress	2-3	Medium
Super-Admin (Developer)	Development team with full system access	6	High

Table 2 – continued

User Class	Description	Count (Pilot)	Technical Skill
Content Reviewer	Human reviewers approving AI-generated content	2-3	High

2.3 Operating Environment

Component	Environment
Frontend	React 18+, TypeScript, modern browsers (Chrome, Firefox, Edge)
Backend	.NET 8, ASP.NET Core, Linux (Fly.io)
Database	PostgreSQL 15+ (single instance, two schemas: `game`, `content`)
AI Service	Python 3.11, FastAPI, Google AI Studio / OpenRouter API
Assets	Cloudflare R2 (zero-egress storage)
Deployment	Fly.io (backend + DB), Vercel (frontend)

2.4 Design & Implementation Constraints

- **Zero budget** for commercial AI video/image generation tools
- **No runtime AI in assessment path** — AI only for side-project task flavor text
- **Pedagogical alignment** — every game element maps to CS111 lecture content
- **Research validity** — pilot study requires controlled comparison group
- **Accessibility** — must function on low-end devices (classroom computers)
- **No persistent WebSocket** — polling acceptable at 50-user scale

2.5 Assumptions & Dependencies

- Students have basic computer literacy (mouse, keyboard, browser)
- Classroom has stable internet (minimum 2 Mbps per student)
- Dr. Amr provides CS111 curriculum mapping and question bank
- Google AI Studio free tier or OpenRouter credits available for AI pipeline
- Fly.io free tier sufficient for pilot (50 concurrent users)

---

3. SYSTEM FEATURES & FUNCTIONAL REQUIREMENTS

3.1 Authentication & Authorization (F-AUTH)

F-AUTH-001: Player Login

Priority: High **Description:** Players authenticate using Class Code and Student ID. **Inputs:** Class Code (string), Student ID (string) **Process:**

1. System validates Class Code exists and is active

2. System hashes Student ID for privacy
3. If new player: create account, initialize save state, return JWT
4. If existing player: return JWT + saved desktop state **Outputs:** JWT access token (1h expiry), JWT refresh token (7d expiry), DesktopState JSON **Error Cases:** Invalid class code, inactive class, malformed student ID

F-AUTH-002: Role-Based Access Control (RBAC)

Priority: High **Description:** Three distinct roles with hierarchical permissions. **Roles:**

- `player`: Can only access own game state
- `admin`: Can view all students in assigned classes, export reports
- `super_admin`: Full system access, user management, configuration **Implementation:** JWT claims include `role` and `class_codes` (for admin). Middleware enforces route-level authorization.

F-AUTH-003: Session Management

Priority: High **Description:** Automatic token refresh, concurrent session handling, logout. **Requirements:**

- Access token expires after 1 hour
- Refresh token expires after 7 days
- Maximum 3 concurrent sessions per account
- Logout invalidates all tokens for that user

F-AUTH-004: Admin & Super-Admin Login

Priority: High **Description:** Separate login portal for administrative users. **Inputs:** Email, password (bcrypt hashed) **Process:** Validate credentials → issue admin JWT with role claim → redirect to appropriate dashboard **Security:** Rate-limited (5 attempts per 15 minutes), IP logging for super-admin

---

3.2 Game Core — LoopOS Desktop (F-DESK)

F-DESK-001: Desktop Environment

Priority: High **Description:** Full-screen simulated desktop OS. **Features:**

- Wallpaper rendering (full-bleed, chapter-specific)
- Desktop icons (draggable, double-click to open)
- Context menu (right-click: Refresh, Sort, Change Wallpaper, System)
- Top bar (menu, clock, system tray)
- Dock (app launcher with running indicators and notification badges)
- Window manager (drag, minimize, maximize, z-index, modal blocking)

F-DESK-002: Boot Sequence

Priority: Medium **Description:** POST simulation → Login → Desktop restore. **States:**

- First boot: Day Zero desktop (messy, personal icons)
- Resume: Exact desktop state from last save (open windows, positions, wallpaper)
- New chapter: Wallpaper transition animation

F-DESK-003: Notification System

Priority: High **Description:** Stacking toast notifications in top-right. **Types:** Info (blue), Warning (amber), Critical (red) **Behavior:** Slide in, auto-dismiss (5s for info/warning, persistent for critical), click to open relevant app **History:** Bell icon dropdown shows last 20 notifications

F-DESK-004: Save & Resume

Priority: High **Description:** Automatic and manual save of complete desktop state. **Auto-save:** Every 30 seconds + after every significant event (choice, gate clear, purchase) **Manual save:** "Sleep" (save and return) or "Shut Down" (save and quit) **Save slots:** 3 slots with timestamps

---

3.3 Narrative Engine — Ink.js Integration (F-NARR)

F-NARR-001: Story Beat Delivery

Priority: High **Description:** Backend serves pre-authored story beats; frontend renders via Ink Bridge. **Beat Structure:**

```
{
  "beatId": "shift1_beat3",
  "shiftId": 1,
  "app": "WhatsUpp",
  "from": "Youssef",
  "delay": 1.5,
  "content": { "text": "...", "choices": [...] },
  "desktopEvent": { "type": "unlockApp", "payload": "LoopCode" }
}
```

F-NARR-002: Choice Presentation

Priority: High **Description:** Four large buttons replace chat input when choices are active. **Behavior:**

- Buttons appear only when beat contains choices
- Player clicks one → choice text appears as Mohamed's message → narrative advances
- No "correct" indicator at time of choice
- Choice tier logged as assessment event

F-NARR-003: Delayed Consequence System

Priority: High **Description:** Choices trigger consequences that fire in future shifts. **Mechanism:**

- On choice submission, backend evaluates tier and queues consequence
- Consequence queue stored per player
- At shift start, system checks for triggered consequences and injects them into narrative flow

F-NARR-004: Desktop Events

Priority: Medium **Description:** Non-app narrative manifestations. **Event Types:**

- `unlockApp`: Add icon to dock
- `changeWallpaper`: Crossfade to new image
- `addIcon`: Animate new desktop icon
- `deleteIcon`: Remove icon (e.g., "Dad's Stuff")
- `glitch`: RGB split + horizontal tear (stress moments)
- `systemModal`: Block all interaction (video calls, promotions)

3.4 Application Ecosystem (F-APP)

F-APP-001: WhatsUpp (Chat)

Priority: High **Features:**

- Chat list (search, conversation rows with unread badges)
- Thread view (message bubbles, read receipts, typing indicator)
- Message types: text, image, voice note (waveform), file attachment
- Group chats (member list, reply threading)
- Choice input (4 buttons replacing text field)

F-APP-002: MailLoop (Email)

Priority: High **Features:**

- Inbox (folders: Inbox, Sent, Loop, Archive, Trash)
- Email list (unread indicators, attachment icons)
- Email detail (rich text body, attachment download)
- Narrative unlocks (reading emails triggers desktop events)

F-APP-003: LoopCode (IDE + Practice)

Priority: High **Features:**

- Two modes: Story Mode (read-only/review) and Practice Mode (interactive)
- Code editor: line numbers, syntax highlighting (C), starter code
- Input modes: Type Mode (free typing) and Assemble Mode (drag-and-drop blocks)
- Action bar: Run (test without submit), Submit (evaluate), Hint

- Output panel: compiler output, test case results
- Progress bar: streak counter, "3 clean in a row" indicator

F-APP-004: Browser / LoopAssist (Hints)

Priority: Medium **Features:**

- LoopAssist chat (player-initiated only)
- Tiered hints: conceptual → specific → directive
- Internal wiki (concept pages, searchable)
- Usage tracking (every hint request logged)

F-APP-005: Files

Priority: Medium **Features:**

- Folder tree (Personal, Loop, References)
- File viewer (PDF, image, text)
- Unlock animation for new files

F-APP-006: Calendar

Priority: Low **Features:**

- Month grid view
- Shift markers (past = checkmark, current = highlighted, future = locked)
- Milestone markers (Tante Layla opening day)

F-APP-007: Terminal

Priority: Low **Features:**

- Functional commands: ls, cd, pwd, git status, build
- Narrative commands (easter eggs)
- Green-on-black monospace aesthetic

F-APP-008: Settings

Priority: Medium **Features:**

- General: text speed, language, time format
- Audio: music/SFX/voice volume sliders
- Display: fullscreen, brightness
- Account: read-only student info, rank display
- Data: Sleep, Shut Down, save slots (Load/Delete)

F-APP-009: Video Call (Modal)

Priority: High **Features:**

- Modal window (blocks all other interaction)
- Static portrait with quality-based rendering (broken/720p/1080p/DSLR)
- Subtitle text with typewriter effect
- Call duration timer (decorative)
- Disabled controls (mute, camera, end call)

3.5 Practice & Assessment (F-PRAC)

F-PRAC-001: Practice Gate

Priority: High **Description:** Mandatory post-shift coding exercises. **Requirements:**

- 5-10 problems per gate
- 20% spaced retrieval from previous chapters
- Exit condition: 3 consecutive correct responses (Ideal or Acceptable)
- Struggle detection: 5+ attempts without streak → help message from Youssef
- Blocking: Cannot proceed to next shift until cleared

F-PRAC-002: Code Evaluation

Priority: High **Description:** Deterministic evaluation of submitted C code. **Process:**

1. Parse submitted code
2. Run against pre-defined test cases (input → expected output)
3. Check rubric criteria (syntax, logic, edge cases, style)
4. Map to tier: Ideal / Acceptable / Debt / Mistake
5. Return tier + specific feedback text

F-PRAC-003: Chapter Recap Gate

Priority: Medium **Description:** Pre-shift knowledge check mixing previous chapter concepts. **Format:** Assemble-the-lines or multiple choice **Purpose:** Spaced retrieval, distributed practice

F-PRAC-004: Stealth Assessment Logging

Priority: High **Description:** Invisible logging of all learning-relevant behavior. **Logged Events:**

- Choice submissions (beatId, choiceId, tier, timestamp)
- Practice attempts (taskId, code, tier, test results, time spent)
- Hint requests (concept, hint level, timestamp)
- Side task submissions (taskId, tier, earnings)

- Desktop interactions (apps opened, time per app, files read)
- Delayed consequence triggers

3.6 Economy System (F-ECON)

F-ECON-001: Income Sources

Priority: Medium **Description:** Multiple income streams. **Sources:**

- Base salary (2,000 → 12,000 EGP based on rank)
- Performance bonus (+500 Acceptable / +1,000 Ideal)
- Side project task reward (500 → 3,000 EGP)
- Bug bounty (+200 EGP for self-found bugs)

F-ECON-002: Expense System

Priority: Medium **Description:** Player spends EGP on upgrades and cosmetics. **Categories:**

- Sahm tiers (Free → Pro → Team → Enterprise)
- Camera upgrades (broken → 720p → 1080p → DSLR)
- Desk items (keyboard, mouse, mug, plant, LED strip)
- Workspace upgrades (second monitor, standing desk)

F-ECON-003: Transaction History

Priority: Low **Description:** Immutable log of all economic activity. **Fields:** amount, type (salary/bonus/side_task/purchase/penalty), description, timestamp

F-ECON-004: Purchase Validation

Priority: Medium **Description:** Prevent overspending, enforce rank requirements. **Rules:**

- Cannot purchase if balance < cost
- Some items require minimum rank
- Sahm upgrade is one-way (cannot downgrade)

3.7 Side Project & Sahm AI (F-SIDE)

F-SIDE-001: Task Generation

Priority: High **Description:** Dynamic side project tasks based on player performance. **Process:**

1. Identify weakest concept from assessment events
2. Select pre-authored template matching concept + rank
3. Call LLM API to fill variable slots (product names, prices, quantities)
4. Validate generated values against constraints

5. Assemble and deliver task to player **Guardrails:** Template skeleton is locked; AI only fills 3-5 slots; validator rejects bad values

F-SIDE-002: Sahm Tier System

Priority: High **Description:** Four-tier AI assistant with escalating capabilities. **Tiers:**

- Free: Error checking, high-level plan (3 bullets), 3 hints/day
- Pro: Detailed plan (7-10 steps), 5-line code snippets in side panel, 10 hints/day
- Team: Module-level plans, 20-line modules with Insert button (requires review), unlimited hints
- Enterprise: System design diagrams, end-to-end generation, predictive hints

F-SIDE-003: Task Submission & Evaluation

Priority: High **Description:** Same evaluation engine as practice gates. **Process:** Submit code → run test cases → apply rubric → return tier + feedback + EGP reward

F-SIDE-004: Task Abandonment

Priority: Medium **Description:** Player can abandon current task. **Penalty:** No reward, -100 EGP time penalty, next task is easier **Cooldown:** 10-minute wait before new task offer

F-SIDE-005: Side Project Narrative Gates

Priority: Medium **Description:** Certain main narrative beats require side project progress. **Example:** Youssef: "I heard you're building a tool. Show me what you've learned." → Player must complete at least 2 side tasks to proceed.

3.8 Promotion System (F-PROMO)

F-PROMO-001: Rank Progression

Priority: Medium **Description:** Five-tier career progression. **Ranks:** Intern → Fresh → Experienced Junior → Senior → Lead **Requirements:**

- Fresh: Complete Shift 3 + clear Chapter 1 recap gate
- Experienced Junior: Complete Capstone (Chapter 1)
- Senior: Complete Chapter 2 Capstone
- Lead: Complete Chapter 3 Capstone

F-PROMO-002: Promotion Ceremony

Priority: Medium **Description:** Multi-channel promotion notification. **Components:**

- System modal: "Promotion: [Old] → [New]"
- MailLoop: HR email with compensation adjustment
- WhatsUpp: Youssef acknowledgment
- Desktop: Wallpaper change to better office

F-PROMO-003: Salary Adjustment

Priority: Medium **Description:** Automatic salary increase on promotion. **Effective:** Next shift after promotion

---

3.9 Inventory & Cosmetics (F-INV)

F-INV-001: Inventory Management

Priority: Low **Description:** Track owned items. **Items:** Camera, keyboard, mouse, desk items, monitors, standing desk

F-INV-002: Desktop Rendering

Priority: Low **Description:** Visual effects based on inventory. **Effects:**

- Camera quality: scanlines, chromatic aberration, pixelation, freeze probability
- Keyboard: typing sound animation
- Mouse: custom cursor
- Desk items: decorative icons on desktop
- LED strip: ambient glow color

F-INV-003: Shop Interface

Priority: Medium **Description:** Modal window for purchasing items. **Features:** Item grid with prices, preview icons, purchase button (disabled if insufficient funds), rank requirements

---

4. EXTERNAL INTERFACE REQUIREMENTS

4.1 User Interfaces

4.1.1 Player Interface (LoopOS)

- **Resolution:** Responsive, minimum 1280×720
- **Input:** Mouse (primary), keyboard (code editor, terminal)
- **Browser:** Chrome 90+, Firefox 88+, Edge 90+
- **Accessibility:** Keyboard navigation for all interactive elements, ARIA labels

4.1.2 Admin Dashboard

- **URL:** `/admin`
- **Layout:** Sidebar navigation + main content area
- **Theme:** Clean, data-dense, chart-heavy

4.1.3 Super-Admin Dashboard

- **URL:** `/super-admin`
- **Layout:** Sidebar navigation + main content + system status bar

- **Theme:** Dark mode default, technical metrics prominent

4.2 Hardware Interfaces

- Standard PC with keyboard, mouse, display
- Minimum 4GB RAM, 2GB storage
- Internet connection (2 Mbps minimum)

4.3 Software Interfaces

Interface	Protocol	Purpose
LLM API (Gemini/OpenRouter)	HTTPS/REST	Runtime task generation
Cloudflare R2	HTTPS/S3	Asset storage and delivery
PostgreSQL	TCP/5432	Primary database
Frontend Backend	HTTPS/REST + JWT	Game state, content, progress

4.4 Communications Interfaces

- All API communication over HTTPS
- JWT Bearer token authentication
- JSON request/response format
- Rate limiting: 100 requests/minute per IP, 10 AI calls/hour per player

---

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance Requirements

Metric	Requirement	Measurement
Page load time	$<$ 3 seconds	From navigation to interactive desktop
API response time	$<$ 200ms (p95)	For non-AI endpoints
AI generation latency	$<$ 3 seconds (p95)	From request to task delivery
Concurrent users	50	Classroom pilot
Save operation	$<$ 500ms	Full state persistence
Code evaluation	$<$ 2 seconds	Sandbox execution + test comparison

5.2 Availability & Reliability

- **Uptime:** 99% during pilot period (classroom hours)
- **Auto-save:** Prevents data loss on browser crash
- **Graceful degradation:** If AI service is down, serve pre-authored fallback tasks
- **Backup:** Daily automated database backups

5.3 Scalability

- Horizontal scaling not required for pilot

- Database designed for future partitioning by class_code
- Asset delivery via CDN (Cloudflare) handles load automatically

5.4 Maintainability

- **Code coverage:** Minimum 70% unit test coverage for backend
- **Documentation:** XML comments on all public APIs, OpenAPI spec
- **Logging:** Structured logs (Serilog) with correlation IDs
- **Monitoring:** Health checks on all services

5.5 Portability

- Frontend: Standard web technologies, no native dependencies
- Backend: .NET 8 (cross-platform), Docker-compatible
- Database: PostgreSQL (widely supported)

5.6 Usability

- **Onboarding:** Interactive tutorial on first boot (desktop navigation, app usage)
- **Help system:** LoopAssist with contextual hints
- **Error messages:** Plain language, actionable, diegetic ("The compiler says…" not "Error 500")
- **Text speed:** Adjustable (slow/medium/fast)

5.7 Localization

- **Primary:** English (interface and content)
- **Future:** Arabic UI localization (RTL support)
- **Content:** Code examples and variable names in English (CS111 is taught in English)

---

6. DATA REQUIREMENTS

6.1 Data Models

See Section 14 of System Design Specification for complete database schema.

Key Entities

- **Player:** Identity, authentication, role
- **Save:** Complete desktop state (JSONB)
- **StoryBeat:** Pre-authored narrative content
- **Choice:** Four options per beat with tier and consequence
- **AssessmentEvent:** Stealth assessment log
- **PracticeTask:** Pre-authored main gate tasks
- **SideTaskTemplate:** Pre-authored skeletons for runtime generation
- **PlayerSideTask:** Runtime-generated task instances
- **PlayerEconomy:** Balance, salary tier, transactions
- **PlayerInventory:** Owned items
- **SahmSubscription:** Tier, hint usage

6.2 Data Retention

- **Player saves:** Retained indefinitely (research data)
- **Assessment events:** Retained indefinitely (anonymized for research)
- **AI audit logs:** Retained for 2 years
- **Transactions:** Retained indefinitely (financial record)
- **Deleted accounts:** Soft delete (flagged, data retained for research)

6.3 Data Export

- **Admin export:** CSV/Excel of class performance metrics
- **Research export:** Anonymized JSON of assessment events for statistical analysis
- **Super-admin export:** Full database dump (encrypted)

---

7. SECURITY REQUIREMENTS

7.1 Authentication & Authorization

- JWT with short expiry (1h access, 7d refresh)
- Password hashing: bcrypt (cost factor 12) for admin accounts
- Role-based access control (RBAC) on all endpoints
- Admin login rate-limited (5 attempts per 15 minutes)
- Super-admin login: additional IP whitelist + email notification

7.2 Data Protection

- Student IDs hashed (SHA-256) before storage
- No personally identifiable information in assessment logs
- HTTPS for all communications
- Database credentials in environment variables, never in code

7.3 Input Validation

- All user inputs sanitized (XSS prevention)
- Code submissions run in sandboxed Docker containers (no network, limited resources)
- File uploads: size limit (10MB), type whitelist (PDF, PNG, JPG)
- AI prompt injection prevention: templated prompts, output validation

7.4 Audit & Logging

- All admin actions logged (who, what, when, IP)
- All AI generation calls logged (prompt, response, latency, cost)
- Failed login attempts logged
- Data export operations logged

7.5 Compliance

- GDPR-style right to data deletion (soft delete + anonymization)
- Research ethics approval (to be obtained from Helwan University)

- Parental consent for minors (if applicable)

---

8. ADMIN & SUPER-ADMIN PANEL REQUIREMENTS

8.1 Admin Panel (Instructor — Dr. Amr)

8.1.1 Authentication (A-AUTH)

A-AUTH-001: Admin login via email/password with 2FA option. **A-AUTH-002:** Session timeout after 30 minutes of inactivity. **A-AUTH-003:** Password reset via email link (expires in 1 hour).

8.1.2 Dashboard (A-DASH)

A-DASH-001: Class overview showing:

- Total enrolled students
- Active players (last 7 days)
- Average completion rate per shift
- Average practice gate attempts
- Distribution of choice tiers (Ideal/Acceptable/Debt/Mistake)

A-DASH-002: Interactive charts:

- Line chart: class progress over time (shifts completed)
- Bar chart: concept mastery distribution (variables, conditionals, loops)
- Pie chart: choice tier distribution
- Heatmap: student × concept mastery matrix

8.1.3 Student Management (A-STUD)

A-STUD-001: Student list with filters (class code, rank, last active, completion status). **A-STUD-002:** Individual student drill-down:

- Profile: student ID (hashed), rank, current shift, total play time
- Progress timeline: shift completion dates, gate clear times
- Choice history: all choices with tiers and timestamps
- Practice performance: attempts per task, tier distribution, hint usage
- Side project: tasks completed, earnings, Sahm tier
- Assessment profile: concept mastery scores (stealth estimate)

A-STUD-003: Student comparison tool: select 2-5 students, compare progress side-by-side.

8.1.4 Analytics & Reports (A-REPT)

A-REPT-001: Class performance report (exportable CSV/Excel/PDF). **Content:**

- Student ID (anonymized)

- Shifts completed
- Average choice tier
- Practice gate pass rate
- Hint usage frequency
- Side project completion
- Current rank
- Estimated concept mastery (per concept)

A-REPT-002: At-risk student identification. **Criteria:**

- > 5 practice gate attempts without clearing
- > 50% Mistake-tier choices
- > 10 hint requests per shift
- No activity for >7 days **Output:** Flagged student list with specific risk factors.

A-REPT-003: Concept difficulty report. **Content:**

- Per-concept pass rate (variables, conditionals, loops)
- Most common error patterns
- Average time to mastery
- Comparison to previous cohorts (if available)

8.1.5 Content Viewer (A-CONT)

A-CONT-001: Read-only view of all story beats, choices, and practice tasks. **A-CONT-002:** Search by concept tag, shift, character, or keyword. **A-CONT-003:** Preview mode: see how a beat renders in the game.

8.1.6 Class Management (A-CLASS)

A-CLASS-001: View class roster (imported from university system or manual entry). **A-CLASS-002:** Activate/deactivate class codes. **A-CLASS-003:** Bulk student enrollment via CSV upload.

8.1.7 Notification Center (A-NOTIF)

A-NOTIF-001: Send broadcast messages to all students in a class (appears as MailLoop email in-game). **A-NOTIF-002:** Send targeted messages to individual students.

8.2 Super-Admin Panel (Development Team)

8.2.1 User Management (SA-USER)

SA-USER-001: Full CRUD on all user accounts (players, admins, super-admins). **SA-USER-002:** Role assignment and modification. **SA-USER-003:** Password reset for any account. **SA-USER-004:**

Account suspension/deactivation. **SA-USER-005:** Impersonation mode: log in as any player to debug issues (logged and auditable).

8.2.2 System Monitoring (SA-MON)

SA-MON-001: Real-time system health dashboard. **Metrics:**

- Server CPU/Memory usage
- Database connection pool status
- API request rate and latency (p50, p95, p99)
- Error rate (4xx, 5xx)
- Active sessions count
- AI service status (up/down, queue depth)

SA-MON-002: Alert configuration. **Triggers:**

- Error rate $> 5\%$
- API latency p95 > 500ms
- Database connections $> 80\%$ of pool
- AI service down > 2 minutes
- Disk space $> 85\%$

SA-MON-003: Log viewer. **Features:**

- Structured log search (by level, correlation ID, endpoint, user)
- Real-time tail (WebSocket)
- Export to CSV

8.2.3 AI Pipeline Monitoring (SA-AI)

SA-AI-001: AI usage dashboard. **Metrics:**

- Total API calls (daily, weekly, monthly)
- Average latency per call
- Cost per day (estimated EGP/USD)
- Success rate (200 vs error responses)
- Validation pass/fail rate
- Fallback usage rate (how often defaults were used)

SA-AI-002: AI output inspector. **Features:**

- Browse all generated tasks with full audit trail
- Filter by template, concept, player, validation status
- Flag problematic outputs for review
- Manual override: regenerate specific task with different parameters

SA-AI-003: Prompt performance analysis. **Metrics:**

- Which templates produce the most validation failures
- Average tokens per prompt/response
- Model comparison (if using multiple models)

8.2.4 Database Browser (SA-DB)

SA-DB-001: Read-only SQL query interface. **Features:**

- Pre-built queries (top players, error patterns, concept coverage)
- Custom SQL with syntax highlighting
- Result export (CSV, JSON)
- Query history

SA-DB-002: Schema viewer. **Features:**

- ER diagram
- Table definitions
- Index usage statistics
- Slow query log

8.2.5 Content Management (SA-CMS)

SA-CMS-001: Story beat editor. **Features:**

- CRUD on story beats
- Choice editor with tier assignment
- Delayed consequence configuration
- Preview in game context
- Version control (track changes, rollback)

SA-CMS-002: Practice task editor. **Features:**

- CRUD on practice tasks
- Test case builder
- Rubric editor (four tiers)
- Starter code editor with syntax highlighting
- Difficulty tagging

SA-CMS-003: Side task template editor. **Features:**

- CRUD on templates
- Slot definition (name, type, constraints)
- LLM prompt editor
- Validation rule builder

- Preview generated task with sample slot values

SA-CMS-004: Shop item editor. **Features:**

- CRUD on shop items
- Cost and requirement configuration
- Icon upload

8.2.6 Configuration Management (SA-CONFIG)

SA-CONFIG-001: Game balance tuning. **Editable parameters:**

- Salary amounts per rank
- Sahm tier costs
- Item costs
- Practice gate exit condition (default: 3 consecutive)
- Hint daily limits per tier
- Side task reward multipliers

SA-CONFIG-002: Feature flags. **Toggleable features:**

- Side project enabled/disabled
- AI generation enabled/disabled (fallback to pre-authored)
- Economy system enabled/disabled
- Promotion system enabled/disabled
- Debug mode (shows tier labels on choices)

SA-CONFIG-003: AI service configuration. **Editable parameters:**

- LLM provider (Gemini / OpenRouter)
- API key rotation
- Model selection
- Temperature setting
- Max tokens
- Rate limits

8.2.7 Security & Audit (SA-SEC)

SA-SEC-001: Admin action audit log. **Content:**

- Timestamp
- Admin user
- Action type (view/export/edit/delete)
- Target resource (student ID, content ID)
- IP address

- User agent

SA-SEC-002: Security event log. **Content:**

- Failed login attempts
- Suspicious activity (unusual API patterns)
- Data export operations
- Permission escalation attempts

SA-SEC-003: IP whitelist management. **Features:**

- Add/remove super-admin access IPs
- View recent login IPs
- Block suspicious IPs

8.2.8 Backup & Maintenance (SA-MAINT)

SA-MAINT-001: Database backup management. **Features:**

- Trigger manual backup
- View backup history
- Restore from backup (confirmation required)
- Download backup files (encrypted)

SA-MAINT-002: Cache management. **Features:**

- View cache statistics
- Clear specific caches
- Warm caches

SA-MAINT-003: Maintenance mode. **Features:**

- Enable/disable maintenance mode
- Custom maintenance message
- Allow admin access during maintenance

8.3 Permission Matrix

Feature	Player	Admin	Super-Admin
Play game	✅	❌	❌
View own progress	✅	❌	❌
View class dashboard	❌	✅ (own classes)	✅ (all)
View individual student	❌	✅ (own classes)	✅ (all)
Export reports	❌	✅ (own classes)	✅ (all)
Send notifications	❌	✅ (own classes)	✅ (all)

Table 6 – continued

Feature	Player	Admin	Super-Admin
Manage class roster	❌	✅ (own classes)	✅ (all)
View content	❌	✅ (read-only)	✅ (full CRUD)
Edit content	❌	❌	✅
Manage users	❌	❌	✅
System monitoring	❌	❌	✅
AI pipeline control	❌	❌	✅
Database browser	❌	❌	✅
Configuration	❌	❌	✅
Security audit	❌	❌	✅
Impersonation	❌	❌	✅
Backup/restore	❌	❌	✅

---

9. AI PIPELINE REQUIREMENTS

9.1 Functional Requirements (F-AI)

F-AI-001: Template-Based Generation

Priority: High **Description:** AI fills variable slots in pre-authored templates, never generating full tasks from scratch. **Slots:** Product name (string), price (float), quantity (int), day (enum) **Constraints:**

- Product names: 1-50 chars, no profanity, clothing/streetwear themed
- Prices: positive float, $<$ 10,000 EGP
- Quantities: positive int, $<$ 1,000
- Days: valid weekday names

F-AI-002: Prompt Construction

Priority: High **Description:** Backend assembles structured prompt from template + player context. **Template:**

```
You are filling variable slots in a C programming exercise template.
CONCEPT: {concept}
DIFFICULTY: {difficulty}
PLAYER CONTEXT: {performance_summary}
SLOTS TO FILL: {slot_list}
RULES: {rules}
RESPOND IN JSON: {json_schema}
```

F-AI-003: Response Validation

Priority: High **Description:** Backend validates LLM output before task assembly. **Checks:**

- JSON parseability

- All required slots present
- Slot values match type constraints
- No harmful content
- Values appropriate for pedagogical context

F-AI-004: Retry & Fallback

Priority: High **Description:** Handle validation failures gracefully. **Process:**

1. First attempt: standard prompt
2. If validation fails: retry with stricter prompt (max 2 retries)
3. If still failing: use pre-authored default values from template
4. Log all failures for human review

F-AI-005: Audit Trail

Priority: High **Description:** Every generation call stored with full context. **Stored:**

- Timestamp
- Player ID
- Template ID
- LLM model name
- Full prompt text
- Full raw response
- Validation result
- Final assembled task
- Latency (ms)

F-AI-006: Cost Control

Priority: Medium **Description:** Monitor and limit API usage. **Limits:**

- Per player: 10 calls/hour
- Per class: 100 calls/hour
- Per day: 1,000 calls total
- Alert at 80% of daily limit

F-AI-007: Model Selection

Priority: Medium **Description:** Configurable LLM provider and model. **Supported:**

- Google Gemini Flash (primary, cheap)
- OpenRouter with any model (fallback)
- Switchable via super-admin configuration

9.2 Non-Functional Requirements (NF-AI)

Metric	Requirement
Latency	$<$ 3 seconds p95 from request to task delivery
Availability	95% (graceful fallback to pre-authored tasks if down)
Cost	$<$ \$5 total for entire pilot (50 students $\times$ ~2500 tasks)
Accuracy	$>$ 95% validation pass rate
Safety	Zero harmful content reaching players

---

10. APPENDICES

Appendix A: Use Case Diagrams

A.1 Player Use Cases

```
[Player]
  ├── Login (Class Code + Student ID)
  ├── Play Shift
  │     ├── Read narrative beats
  │     ├── Make choices
  │     ├── Write code in LoopCode
  │     ├── Complete practice gate
  │     └── Receive feedback
  ├── Manage Side Project
  │     ├── Request task from Sahm
  │     ├── Write side project code
  │     ├── Submit task
  │     └── Earn EGP
  ├── Manage Economy
  │     ├── View balance
  │     ├── Purchase Sahm upgrade
  │     ├── Purchase desk items
  │     └── Upgrade camera
  ├── View Progress
  │     ├── Check rank
  │     ├── View calendar
  │     └── Read past emails
  └── Configure Settings
        ├── Adjust text speed
        ├── Change audio volumes
        └── Manage save slots
```

A.2 Admin Use Cases

```
[Admin]
  ├── Login (Email + Password)
```

```
  ├── View Dashboard
  │     ├── Class overview
  │     ├── Progress charts
  │     └── Concept mastery heatmap
  ├── Manage Students
  │     ├── View student list
  │     ├── Drill into individual student
  │     └── Compare students
  ├── Generate Reports
  │     ├── Class performance report
  │     ├── At-risk students
  │     └── Concept difficulty report
  ├── View Content
  │     ├── Browse story beats
  │     ├── Search content
  │     └── Preview in game
  ├── Manage Classes
  │     ├── View roster
  │     ├── Activate/deactivate codes
  │     └── Bulk enrollment
  └── Send Notifications
        ├── Broadcast to class
        └── Targeted to student
```

A.3 Super-Admin Use Cases

```
[Super-Admin]
  ├── All Admin Use Cases
  ├── Manage Users
  │     ├── CRUD all accounts
  │     ├── Assign roles
  │     └── Impersonate player
  ├── Monitor System
  │     ├── View health dashboard
  │     ├── Configure alerts
  │     └── View logs
  ├── Manage AI Pipeline
  │     ├── Monitor usage and cost
  │     ├── Inspect generated tasks
  │     └── Configure model/settings
  ├── Manage Content
  │     ├── CRUD story beats
  │     ├── CRUD practice tasks
  │     ├── CRUD side task templates
  │     └── Version control
  ├── Manage Configuration
  │     ├── Tune game balance
  │     ├── Toggle feature flags
  │     └── Edit AI parameters
  ├── Security & Audit
  │     ├── View admin action log
```

```
│     ├── View security events
│     └── Manage IP whitelist
└── Maintenance
      ├── Database backup/restore
      ├── Cache management
      └── Maintenance mode
```

Appendix B: API Endpoint Summary

Player-Facing Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/login	None	Player login
GET	/api/content/beats/{shiftId}	Player	Story beats
POST	/api/progress/choice	Player	Submit choice
POST	/api/progress/practice	Player	Submit practice
POST	/api/progress/save	Player	Save state
GET	/api/progress/state	Player	Load state
GET	/api/economy/balance	Player	Balance
GET	/api/sahm/task	Player	Get side task
POST	/api/sahm/task/submit	Player	Submit side task
POST	/api/sahm/task/abandon	Player	Abandon task
POST	/api/sahm/hint	Player	Request hint
POST	/api/sahm/upgrade	Player	Upgrade Sahm
GET	/api/shop/items	Player	Shop catalog
POST	/api/shop/purchase	Player	Purchase item
GET	/api/inventory	Player	Inventory

Admin Endpoints

Method	Endpoint	Auth	Description
POST	/api/admin/auth/login	None	Admin login
GET	/api/admin/dashboard	Admin	Class overview
GET	/api/admin/students	Admin	Student list
GET	/api/admin/students/{id}	Admin	Student detail
GET	/api/admin/reports/performance	Admin	Performance report
GET	/api/admin/reports/at-risk	Admin	At-risk students
POST	/api/admin/notifications	Admin	Send notification

Table 9 – continued

Method	Endpoint	Auth	Description
GET	/api/admin/content/beats	Admin	Content browser

Super-Admin Endpoints

Method	Endpoint	Auth	Description
POST	/api/super/auth/login	None	Super-admin login
GET	/api/super/users	Super	All users
PUT	/api/super/users/{id}/role	Super	Change role
GET	/api/super/system/health	Super	Health metrics
GET	/api/super/system/logs	Super	Log viewer
GET	/api/super/ai/usage	Super	AI usage stats
GET	/api/super/ai/outputs	Super	Generated tasks
POST	/api/super/ai/regenerate	Super	Regenerate task
GET	/api/super/db/query	Super	SQL browser
GET	/api/super/db/schema	Super	Schema viewer
GET	/api/super/content/beats	Super	Content CRUD
POST	/api/super/content/beats	Super	Create beat
PUT	/api/super/content/beats/{id}	Super	Update beat
DELETE	/api/super/content/beats/{id}	Super	Delete beat
GET	/api/super/config	Super	View config
PUT	/api/super/config	Super	Update config
POST	/api/super/backup	Super	Trigger backup
POST	/api/super/maintenance	Super	Toggle maintenance

Appendix C: Glossary

Term	Definition
Assemble Mode	Drag-and-drop code block interface for practice problems
Beat	Single unit of narrative delivery (message, email, notification)
Choice Tier	Quality classification: Ideal, Acceptable, Debt, Mistake
Consequence Queue	Deferred narrative effects triggered by past choices
Debt	Functional but fragile code that causes future problems

Table 11 – continued

Term	Definition
Diegetic UI	Interface elements that exist within the game world
ECD	Evidence-Centered Design —assessment framework
Epistemic Conflict	Moment when a learner realizes their understanding is wrong
Guarded Runtime AI	AI that fills slots in locked templates, not free generation
Impasse	Point in problem-solving where current knowledge is insufficient
Legitimate Peripheral Participation	Learning by gradually participating in a community
Mistake	Conceptually wrong choice that fails immediately
Practice Gate	Mandatory post-shift coding exercise
Sahm	AI coding assistant for side project tasks
Shift	Narrative workday unit containing beats and practice
Side Project	Parallel self-directed coding tasks (Khamis brand)
Stealth Assessment	Embedded learning measurement through behavior
Type Mode	Free typing code editor interface

Appendix D: Revision History

Version	Date	Author	Changes
0.1	2026-08-19	SHIFT Team	Initial draft
0.2	2026-08-21	SHIFT Team	Added side project, economy, Sahm AI
0.3	2026-08-22	SHIFT Team	Added runtime AI, admin/ super-admin panels
1.0	2026-08-22	SHIFT Team	Complete SRS for committee submission